✅ What is an Index in SQL?

An **Index** is a special database structure that:

Helps the database find data faster without scanning the entire table.

📖 Think of it like:

- The index page of a book
- Instead of reading the whole book, you jump directly to the page

---

🎯 Why Indexes Are Important

Indexes:

✓ Speed up SELECT queries
✓ Improve JOIN performance
✓ Make WHERE filtering faster
✓ Improve ORDER BY sorting
✓ Reduce disk I/O
✓ Ensure uniqueness (Unique Index)

⚠️ Important Notes

- Primary Key automatically creates an index
- Unique constraint automatically creates an index
- Indexes improve **read speed**
- But slightly slow down **INSERT, UPDATE, DELETE**

🔥 Types of Indexes

There are 3 main types:

1️⃣ Single Column Index
2️⃣ Multi Column Index
3️⃣ Unique Index

1️⃣ Single Column Index

👉 **Created on one column only.**

Used when:

- You frequently search/filter by one column

Syntax:

CREATE INDEX index_name

ON table_name(column_name);

Example:

CREATE INDEX idx_product_id

ON Sales(product_id);

✓ Now queries using:

WHERE product_id = 101

Will run faster.

2️⃣ Multi-Column Index

👉 **Created on two or more columns.**

Used when:

- Queries filter using multiple columns together

Syntax:

CREATE INDEX index_name

ON table_name(column1, column2);

Example:

CREATE INDEX idx_product_quantity

ON Sales(product_id, quantity);

✓ Speeds up queries like:

WHERE product_id = 101 AND quantity = 5;

⚠️ Important Rule (Very Important for Interviews)

For:

(product_id, quantity)

Index works for:

- product_id
- product_id + quantity

But NOT for:

- quantity alone

👉 Order of columns matters!

3️⃣ Unique Index

👉 **Ensures no duplicate values.**

Used for:

- Email
- Employee ID
- Customer ID

Syntax:

```
CREATE UNIQUE INDEX index_name
ON table_name(column_name);
```

Example:

```
CREATE UNIQUE INDEX idx_unique_employee_id
ON Sales(customer_id);
```

Now if you try:

```
INSERT INTO Sales (sale_id, product_id, quantity, customer_id)
VALUES (6, 105, 4, 201);
```

❌ Error if 201 already exists.

📊 Viewing Indexes

To see all indexes in a table:

```
SHOW INDEXES FROM Sales;
```

👉 Displays:

- Index name
- Column
- Type
- Whether unique or not

🗑 Removing an Index

Indexes take space and slow write operations.

If not needed:

Syntax:

DROP INDEX index_name ON table_name;

Example:

DROP INDEX idx_product_quantity ON Sales;

✓ Table data remains safe.

🔄 Rebuilding an Index

Over time indexes can become fragmented.

Rebuild to optimize performance.

Syntax:

ALTER INDEX index_name

ON table_name

REBUILD;

Example:

ALTER INDEX idx_product_id

ON Sales

REBUILD;

✏ Renaming an Index

SQL doesn't directly support rename in some systems.

Example (SQL Server):

EXEC sp_rename

'idx_product_quantity',

'idx_prod_qty',

'INDEX';

🚀 How Index Improves Performance

Without Index:
👉 Full Table Scan
👉 Checks every row

With Index:
👉 Uses B-Tree structure
👉 Directly jumps to matching rows

📉 Drawbacks of Indexes

Too many indexes can:

❌ Slow INSERT
❌ Slow UPDATE
❌ Slow DELETE
❌ Take extra storage

🧠 When to Create an Index?

Create index on columns used in:

✓ WHERE
✓ JOIN
✓ ORDER BY
✓ GROUP BY
✓ Frequently searched columns

❌ When NOT to Use Index?

Don't use index on:

- Small tables
- Columns with few unique values (e.g., gender: M/F)
- Columns rarely used in queries

📊 Index Types (Advanced Knowledge)

Most databases use:

- B-Tree Index (default)
- Clustered Index
- Non-Clustered Index
- Full-Text Index

- Composite Index

🧠 Easy Memory Trick

Index = Shortcut

No Index → Walk entire road
With Index → Use Google Maps shortcut

🎯 Final Summary

Indexes:

✓ Speed up read operations
✓ Improve query performance
✓ Reduce full table scans
✓ Ensure uniqueness

But:

⚠ Slow down write operations
⚠ Use carefully